
ENGENHARIA DE COMPUTAÇÃO

Disciplina: Engenharia de Software

KA 04 — Construção de Software

(Software Construction)

Guia de Padrões de Codificação e Construção Segura

Instituição de Ensino	UNOESC
Professor(a) Responsável	Odemir Moreira da Mata Jr.
Disciplina	Engenharia de Software
Turma / Semestre	JBA620-6
Integrantes do Grupo	Ana Clara Viganó Prestes de Oliveira, Davi Antônio Basotto Minati, Elian Baldasso Pereira Duarte, Jasmine Masson Alves, Lucas Eduardo Heydt, Vitória Aparecida Vendausen
Data de Entrega	24/06/2026

| Histórico de Versões

Versão	Data	Autor(es)	Descrição	Revisado por
v1.0	01/05/2026	Davi Minati	Elaboração do guia de codificação e padrões.	Lucas Heydt

1. Introdução

| 1.1 Propósito

Este guia estabelece os padrões de construção de software para o SisConsult, garantindo a manutenibilidade, a legibilidade e, sobretudo, a segurança e privacidade exigidas pela arquitetura (Privacy by Design)

| 1.2 Tecnologias Alvo (conforme KA 2)

Backend: Node.js com NestJS (TypeScript).

Frontend: Single Page Application (SPA) com React e TypeScript.

Persistência: PostgreSQL 15.

Comunicação: API RESTful (HTTPS).

2. Convenções de Nomenclatura

| 2.1 Classes e Interfaces

Exemplo correto (✓): class PatientController, interface CreateAppointmentDto

Exemplo incorreto (✗): class controller_paciente, interface IAppointment (evitar prefixo 'I')

| 2.2 Métodos e Funções

Exemplo correto (✓): findPatientById(id: string), calculateAppointmentSlots()

Exemplo incorreto (✗): Buscar_Paciente(), get_data_consulta()

| 2.3 Variáveis e Constantes

As variáveis locais devem seguir o padrão camelCase, iniciando com letra minúscula e utilizando letras maiúsculas para separar palavras subsequentes (ex.: patientName, appointmentDate). Constantes de configuração e valores fixos globais devem utilizar o padrão UPPER_SNAKE_CASE, com todas as letras maiúsculas e palavras separadas por underline (ex.: AES_ENCRYPTION_KEY, MAX_LOGIN_ATTEMPTS).

2.4 Arquivos e Pacotes/Módulos

Os arquivos do projeto devem utilizar o padrão kebab-case, com palavras separadas por hífen e todas as letras minúsculas (ex.: medical-record.service.ts, patient-controller.ts). Esse padrão facilita a organização dos módulos e mantém a consistência na estrutura do sistema.

2.5 Tabelas e Colunas do Banco de Dados

As tabelas e colunas do banco de dados PostgreSQL devem seguir o padrão snake_case, utilizando letras minúsculas e separação por underline (ex.: medical_records, patient_id,

appointment_date). Essa convenção melhora a legibilidade das consultas SQL e mantém compatibilidade com as boas práticas de bancos relacionais.

3. Organização do Código-Fonte

3.1 Estrutura de Pacotes / Módulos

src/modules/

Diretório principal que organiza o sistema em módulos independentes, seguindo uma arquitetura modular. Cada módulo concentra suas próprias regras de negócio, controladores, serviços, entidades e repositórios, facilitando a manutenção, escalabilidade e reutilização do código.

patient/

Módulo responsável pelo gerenciamento de pacientes. Implementa operações de cadastro, consulta, atualização e exclusão de dados demográficos, além das validações necessárias para garantir a integridade das informações dos usuários do sistema.

appointment/

Módulo responsável pelo gerenciamento de consultas e agendamentos. Controla a criação, alteração e cancelamento de consultas, além do relacionamento entre pacientes e médicos, verificando disponibilidade de horários e regras de agendamento.

ehr/ (*Electronic Health Record – Prontuário Eletrônico*)

Módulo responsável pelo gerenciamento dos prontuários médicos. Armazena e controla informações clínicas, histórico de consultas, diagnósticos, prescrições, exames e demais registros relacionados ao atendimento do paciente. Por conter dados sensíveis, exige mecanismos adicionais de segurança, controle de acesso e rastreabilidade das operações realizadas.

3.2 Separação por Camadas

Seguir rigorosamente o padrão adotado na arquitetura: **Controller** (interface)
—>**Service** (negócio) —> **Repository** (persistência)

3.3 Critério de Complexidade Ciclomática Máxima

Limite de **10** por função. Métodos com alta complexidade devem ser refatorados em sub-funções para facilitar testes e reduzir erros.

4. Padrões de Comentários e Documentação

4.1 Documentação de Classes e Métodos (JavaDoc / JSDoc)

Obrigatório para métodos de serviço e controllers:

```
/**
```

```
 * @param id UUID do paciente
```

```
 * @returns Dados criptografados do prontuário
```

```
 */
```

| 4.2 Comentários Inline — quando usar e quando não usar

Usar apenas para explicar "porquês" de decisões complexas de criptografia ou segurança, nunca o "quê" (que o código deve expressar por si só).

5. Tratamento de Exceções e Erros

| 5.1 Padrão de Tratamento de Exceções

| 5.2 Logging de Erros — regras de mascaramento de dados pessoais

| 6.1 Validação e Sanitização de Entradas

| 6.2 Práticas para Dados Sensíveis de Saúde

6.3 Uso de Bibliotecas Externas

| 6.4 Gerenciamento de Segredos e Credenciais

7. Checklist de Revisão de Código (Code Review)

Tabelas e Formulários

| Checklist de Revisão de Código (Code Review)

#	Item de Verificação	Obrigatório?	Observação
1	Lógica de negócio está isolada no Service?	Sim	Evitar lógica pesada na Controller.
2	Dados sensíveis estão sendo mascarados em logs?	Sim	Conformidade com regra LGPD 5.2.
3	O princípio de menor privilégio (RBAC) foi aplicado?	Sim	Validar permissões do endpoint.
4	Criptografia AES-256 aplicada no PEP?	Sim	Dados sensíveis devem ser cifrados.
5	Validação de entrada via DTOs configurada?	Sim	Impedir injeção de dados malformados.
6	Não há segredos (senhas/chaves) hardcoded?	Sim	Utilizar apenas variáveis de ambiente.

7	Complexidade ciclomática está abaixo de 10?	Sim	Refatorar se exceder o limite.
8	Cobertura de testes unitários está adequada?	Sim	Mínimo exigido para o módulo novo.

| Checklist de Construção Segura

#	Prática de Segurança	Aplicada?	Evidência / Como Verificar
1	Sanitização de dados de entrada	☐	Validação via class-validator nos DTOs.
2	Autenticação via JWT assinado	☐	Validação do token na camada de Auth.
3	Cifragem de dados em repouso (AES-256)	☐	Verificação do Encryption Component.
4	Auditoria de dependências (npm audit)	☐	Relatório de segurança do build.
5	Proteção contra SQL Injection via ORM	☐	Uso de queries parametrizadas no Repository.
6	Mascaramento de dados em logs	☐	Testes de carga com logs de auditoria.
7	Uso estrito de HTTPS/TLS	☐	Configuração do Nginx no container de proxy.
8	Segregação de logs (Imutabilidade)	☐	Isolamento do banco Security Log DB.

| Observações e Referências Consultadas

SWEBOK v4.0 (IEEE, 2024): Capítulo sobre *Software Construction*.

Lei Geral de Proteção de Dados (LGPD), Lei 13.709/2018: Requisitos de segurança e tratamento de dados sensíveis.